

Manual Técnico

Software De Administración y Monitoreo Para Un Bote De Reciclaje Inteligente Basado En Inteligencia Artificial y Computación En La Nube

Valentina Andrade

Tomás De Andreis

Catalina Gutiérrez

Camilo Millán

Mateo Patiño

Asesor

Ricardo Fonseca

Universidad Sergio Arboleda

Escuela de Ciencias Exactas e Ingenierías

Ciencias de la Computación e IA

2025

Tabla de Contenido

- 1. Introducción4
 - 1.1. Objetivo..... 4
 - 1.2. Alcance Del Proyecto 4
 - 1.3. Público Objetivo 5
- 2. Descripción General Del Sistema7
 - 2.1. Flujo De Funcionamiento 7
 - 2.2. Componentes Principales..... 8
- 3. Requerimientos Del Sistema.....11
 - 3.1. Requerimientos Funcionales 11
 - 3.2. Requerimientos No Funcionales 14
 - 3.3. Requerimientos de Hardware..... 15
 - 3.4. Requerimientos De Software 16
- 4. Diagrama De Arquitectura22
 - 4.1. Diagrama general de arquitectura 23
 - 4.1.1. Capa De Firmware 24
 - 4.1.2. Capa De La Nube (AWS Reference Architecture) 25
 - 4.1.3. Capa De Presentación (Frontend) 28
 - 4.2. Diagrama de casos de uso 31
 - 4.3. Diagrama de clases UML..... 32
 - 4.3.1. Clases Identificables en el Proyecto 32
 - 4.4. Diagrama ERD (Entidad-Relación) 36
 - 4.5. Diagrama De Actividades 37
- 5. Bocetos Página web42

6. Cronograma Gantt.....	46
--------------------------	----

1. Introducción

Este proyecto desarrolla un sistema automático de clasificación de residuos que integra hardware embebido y servicios en la nube. En el dispositivo (a definir exacto) se conectan cámara RGB, sensores (a definir modelos exactos) e iluminación para capturar la evidencia del residuo. Un modelo de visión por computador ejecuta la inferencia en tiempo real y determina la clase (reciclable o no reciclable). Con el resultado se activa el mecanismo de depósito y se registra el evento (fecha/hora, clase, confianza, sensorística, imagen). Cuando hay internet, el equipo sincroniza vía una API: envía metadatos a una base de datos relacional y almacena las imágenes en un repositorio de objetos. Una segunda API alimenta una plataforma web (dashboard) con métricas operativas: volumen por clase, precisión del modelo y alertas (atascos, contenedor lleno, fallos de red). Tras confirmarse la subida, se elimina el respaldo local para evitar saturar el almacenamiento del dispositivo.

1.1. Objetivo

Automatizar la clasificación de residuos mediante IA, sensores y conectividad en la nube, mejorando la separación en la fuente, reduciendo la manipulación manual y generando datos confiables para análisis y toma de decisiones en la universidad.

1.2. Alcance Del Proyecto

Incluye (sí hace):

- Diseño e integración de hardware (cámara, sensores, control de iluminación, actuadores) y software embebido para captura, inferencia y logging.
- Modelo de IA optimizado para dispositivo (CNN/transfer learning).

- API de ingestión en la nube, almacenamiento de eventos en RDS e imágenes en S3; borrado local una vez confirmada la sincronización.
- Dashboard web que consume **otra API** para métricas: volumen/clase de residuos, tasas de acierto, uso por período, errores.

Exclusiones (no hace por ahora):

- Tratamiento/compactación avanzada ni logística de recolección.
- Gestión de múltiples ítems simultáneos (solo un residuo por ciclo).
- Manejo funcional del “contenedor lleno” (se documentan recomendaciones, no se automatiza).
- Clasificación de orgánicos: se redirigen a “no aprovechables” para evitar contaminación.
- Limpieza automática de cámara (solo detección/alerta por obstrucción).

1.3. Público Objetivo

- Comunidad universitaria (estudiantes, docentes y personal administrativo): facilita la separación correcta en la fuente, reduce errores por señalización y retroalimentación visual, y mejora la experiencia de disposición.
- Personal de aseo y recicladores aliados: disminuye la contaminación de las corrientes (menos rechazo), optimiza recorridos y tiempos de recolección, y aumenta la recuperación de materiales aprovechables.

- Oficina de Sostenibilidad / Gestión Ambiental: obtiene trazabilidad y métricas (por punto, franja horaria y material) para reportes internos, cumplimiento normativo y metas de economía circular.
- Dirección y Finanzas: acceso a indicadores para reducir costos de disposición final, priorizar inversiones (p. ej., puntos con mayor impacto) y sustentar decisiones con datos.

2. Descripción General Del Sistema

2.1. Flujo De Funcionamiento

El clasificador inicia con la detección de un residuo, seguida de la captura de su imagen para que el sistema de inteligencia artificial lo clasifique como aprovechable o no aprovechable. Según el resultado, se ejecuta una acción mecánica que direcciona el residuo al contenedor correspondiente, se registran los datos de la transacción, se sincronizan con una plataforma central y finalmente se monitorean las métricas del sistema en tiempo real para su análisis y seguimiento.

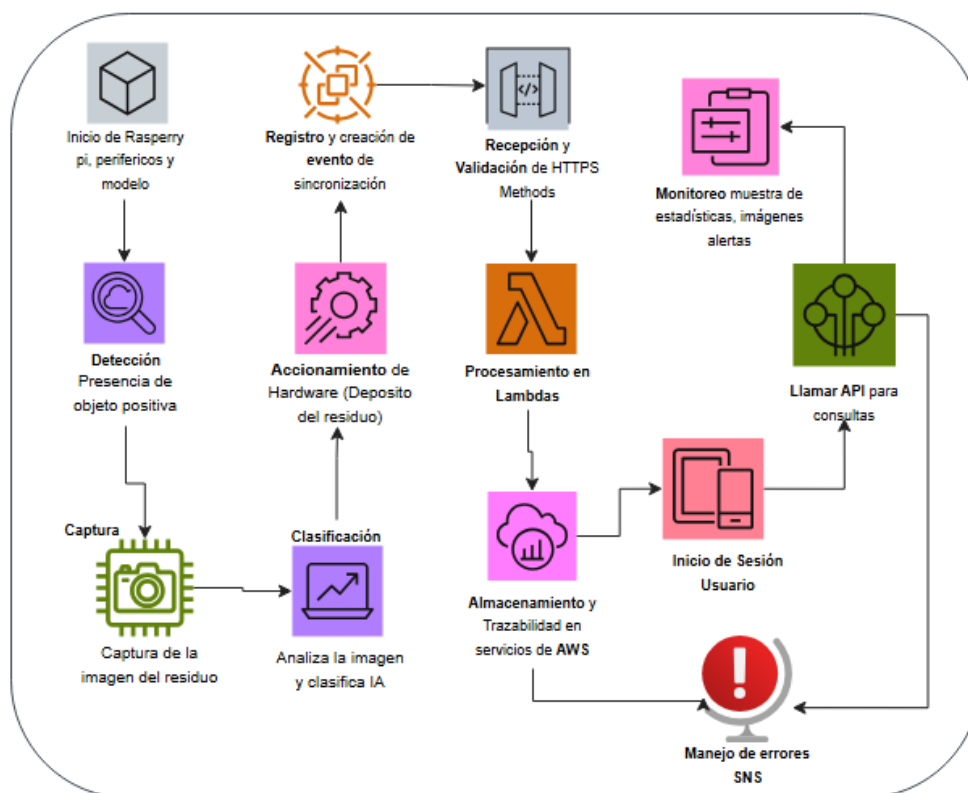


Figura 1 Flujo de funcionamiento básico

2.2. Componentes Principales

- **Hardware:** Los componentes físicos que se implementaran en el proyecto constan de:
 - **Raspberry Pi:** Es el centro de control de todo el sistema. Se encarga de:
 - Ejecutar el software de clasificación por imagen
 - Recibir datos de los otros componentes.
 - Tomar la decisión de qué tipo de basura es.
 - Enviar órdenes a los servomotores para que actúen.
 - **Cámara + Iluminación LED:**
 - La cámara toma una fotografía del objeto de basura que hay en la zona de análisis.
 - La iluminación LED garantizar que la imagen sea clara y consistente, sin sombras ni reflejos engañosos, mejorando la precisión de la clasificación.
 - **Sensor Ultrasónico, Detector de Presencia.**
 - Este sensor se coloca apuntando a la zona donde se deposita la basura.
 - Su función es detectar cuándo un objeto ha sido colocado frente a la cámara. Cuando mide una distancia corta, le avisa a la Raspberry Pi
 - Activando la secuencia y comenzando el análisis.
 - **Servomotores y Mecanismo de Movimiento:**
 - Una vez que la Raspberry Pi ha decidido la categoría del residuo, activa los servomotores.
 - Estos motores son los que generan el movimiento físico

- **Protoboard:**
 - Placa de pruebas que permite conectar de manera temporal y fácil los componentes electrónicos a la Raspberry Pi, usando cables, sin necesidad de soldar.
- **Firmware:** El firmware funciona como el sistema central de operaciones que gestiona y coordina a todos los componentes del proyecto. Su funcionamiento principal se basa en atender las solicitudes de los clientes o usuarios del sistema. Cuando un cliente interactúa con el clasificador de basuras, el firmware recibe esta petición y se convierte en el responsable de orquestar la secuencia completa. Esto implica activar los sistemas de percepción como la cámara y el sensor, procesar los datos para identificar el tipo de residuo, y finalmente ejecutar la acción física correspondiente a través de los servomotores y el mecanismo de movimiento asegurando que cada interacción del usuario culmine con un ciclo completo de clasificación inteligente.
- **Backend (Nube):** Gestionará la lógica, los datos y las comunicaciones de manera segura y escalable. Cuando la Raspberry Pi necesite procesar una clasificación, no lo hace internamente, sino que actúa como un cliente que realiza una petición a una API REST alojada en API Gateway. Esta puerta de enlace recibe la solicitud y la dirige de manera inteligente a una Función Lambda específica, esta se activa únicamente cuando se le solicita una clasificación, ejecutando el modelo de inteligencia artificial para analizar la imagen y determinar el tipo de residuo. Para ello, puede interactuar con otros servicios: puede almacenar la imagen en S3 y guardar un registro de la transacción en la base de datos RDS. Finalmente, una vez completado el procesamiento, el backend envía una

respuesta clara con el resultado de la clasificación de vuelta a la Raspberry Pi a través de la misma API, permitiéndole al dispositivo físico activar los servomotores y realizar la separación física del residuo.

- **Frontend:** Un dashboard accesible desde cualquier navegador que permitirá visualizar el funcionamiento del clasificador y tener control sobre sus operaciones, esta interfaz web se utilizará para mostrar las estadísticas de clasificación, como los tipos de residuos detectados, cantidades procesadas y eficiencia del sistema. Los operadores podrán ver el historial de actividad, generar reportes de operación y recibir alertas cuando ocurran eventos importantes, como cuando se detecte un error en el mecanismo.

3. Requerimientos Del Sistema

3.1. Requerimientos Funcionales

ID	Requerimiento	Descripción
RF1	Detección de objeto	El sistema deberá detectar la presencia de un residuo mediante un sensor ultrasónico y activar automáticamente la cámara y una lámpara auxiliar para mejorar la iluminación.
RF2	Captura de imagen	La Raspberry Pi deberá capturar al menos un frame del residuo, almacenarlo temporalmente en el sistema de archivos local y enviarlo para clasificación.
RF3	Clasificación de residuo mediante IA	El sistema deberá ejecutar un modelo de inteligencia artificial (TensorFlow Lite) entrenado para determinar si el residuo es reciclable o no reciclable.
RF4	Registro local de eventos	El software deberá registrar localmente cada evento con los siguientes datos: event_id (UUID), timestamp, clase predicha, confianza, duración del proceso y errores detectados.
RF5	Gestión de errores locales	El sistema deberá detectar y registrar errores locales (sin conexión WiFi, sensor fallando, cámara obstruida, atasco mecánico) y marcarlos con un error_flag en el evento.

ID	Requerimiento	Descripción
RF6	Sincronización con la nube (AWS)	Cuando haya conexión WiFi disponible, el sistema deberá sincronizar automáticamente los eventos pendientes enviando imágenes y metadatos a AWS.
RF7	Solicitud de URLs pre-firmadas (S3)	El sistema deberá realizar una solicitud a la API AWS Lambda para obtener URLs pre-firmadas que permitan subir las imágenes al bucket S3 sin exponer credenciales en la Raspberry Pi.
RF8	Subida de imágenes a S3	Usando las URLs pre-firmadas, el sistema deberá subir cada imagen del evento a Amazon S3 y guardar la URL resultante en el registro local y en los metadatos a enviar a la base de datos.
RF9	Envío de metadatos a la API AWS	El sistema deberá enviar, vía HTTP POST, los metadatos del evento al endpoint correspondiente.
RF10	Almacenamiento en base de datos RDS	La función Lambda en AWS deberá procesar los metadatos recibidos y guardarlos en una base de datos relacional (RDS PostgreSQL o MySQL).
RF11	Gestión de sincronización local	Una vez confirmada la subida de un evento, el sistema deberá marcarlo como sincronizado en un archivo index.json y eliminar los archivos asociados (imagen y log) del almacenamiento local.

ID	Requerimiento	Descripción
RF12	Monitoreo y métricas en la nube	La API deberá permitir consultar las métricas de uso: número de residuos procesados por día, porcentaje de reciclables, tiempos promedio de procesamiento y errores.
RF13	Visualización en Dashboard Web	El sistema deberá contar con una aplicación web que muestre las estadísticas almacenadas en RDS, las imágenes clasificadas y los estados de los eventos sincronizados.
RF14	Revisión manual (QA)	Los revisores podrán acceder desde el Dashboard para validar si la clasificación fue correcta o incorrecta; esta validación actualizará los datos de la clasificación en la base de datos RDS.
RF15	Gestión de errores en la nube	La API/Lambda deberá guardar los errores recibidos en una tabla o campo JSONB, y enviar alertas (por ejemplo, vía AWS SNS o CloudWatch) cuando se detecten errores críticos o repetitivos.
RF16	Recuperación ante fallos de sincronización	Si una subida falla (por pérdida de red o error HTTP), el sistema deberá reintentar automáticamente la sincronización al restablecer la conexión.

3.2. Requerimientos No Funcionales

ID	Categoría	Requerimiento
RNF1	Conectividad	El sistema deberá poder operar sin conexión WiFi y sincronizar datos posteriormente.
RNF2	Seguridad	La transferencia de datos e imágenes deberá realizarse mediante HTTP y URLs prefirmadas.
RNF3	Escalabilidad	La API y base de datos deberán poder soportar múltiples botes conectados simultáneamente.
RNF4	Confiabilidad	Los eventos no deberán perderse en caso de reinicio del sistema; se deberá usar almacenamiento persistente local.
RNF5	Compatibilidad	El dashboard deberá ser accesible desde navegadores modernos (Chrome, Edge, Firefox).
RNF6	Diseño Responsivo	La aplicación Web deberá poder visualizarse en distintos tipos de pantalla.
RNF7	Mantenibilidad	El código deberá estar modularizado y documentado.
RNF8	Disponibilidad	El sistema de nube deberá tener una disponibilidad mínima del 99%.
RNF9	Privacidad de datos	Las imágenes deberán almacenarse únicamente con fines de clasificación y validación, sin información personal.

3.3. Requerimientos de Hardware

Para el prototipo se consideran dos opciones de controlador principal se usará solo una según lo que resulte más adecuado y fácil durante el desarrollo y un conjunto de periféricos comunes. Todo está sujeto a cambios según pruebas, disponibilidad y el alcance final.

Opción A — Raspberry Pi con Wi-Fi (controlador único):

- Ejecuta la IA local, orquesta sensores/actuadores y sincroniza con la nube.
- Periféricos:
 - Servos ESG5010 (accionamiento de compuertas).
 - Sensor ultrasónico HC-SR04 (presencia/llenado).
 - Regulador/fuente: riel separado 5–6 V para servos y 5 V para la Pi (GND común).

Opción B — ESP32 (15 pines) o ESP32-CAM (controlador único):

- Controla sensores/actuadores y conectividad; la ESP32-CAM integra cámara para captura básica.
- Periféricos:
 - Servos ESG5010 (PWM desde ESP32).
 - Sensor ultrasónico HC-SR04.
 - Regulador/fuente: 5–6 V para servos y 5 V/3.3 V para la ESP32 (GND común).

Notas generales:

- El módulo cámara ESP32-CAM solo aplica en Opción B; con Raspberry se usaría una cámara compatible con la Pi.
- Se recomienda fuente separada para servos y mantener tierra común en todo el sistema.
- Esta selección (A o B) y los componentes pueden ajustarse según pruebas de estabilidad, facilidad de integración y desempeño real del prototipo.

Nota: esta lista puede modificarse (equivalentes, cantidades o especificaciones) según pruebas, disponibilidad, consumo eléctrico, precisión requerida y el alcance final del prototipo.

3.4. Requerimientos De Software

- Firmware

El firmware es el código que controla el comportamiento del sistema físico en la Raspberry Pi, ejecutando el ciclo principal planteado donde se hace desde la detección del objeto hasta el monitoreo de los resultados

El firmware se implementa en Python debido a su versatilidad, compatibilidad con hardware (mediante librerías como RPi.GPIO, picamera, opencv-python, tflite-runtime), y su orientación multiparadigma.

El enfoque utilizado es la Programación Orientada a Objetos (POO), lo que permite dividir el sistema en clases modulares, cada una con una responsabilidad clara dentro del flujo de

funcionamiento y facilitando la abstracción de hardware, reutilización de código y escalabilidad del sistema.

- Librerías: WiFi.h, ESP32HTTPClient.h, ArduinoJson.h, Servo.h.

En caso de usar un microcontrolador auxiliar (como un ESP32) para tareas de control, se pueden emplear librerías equivalentes al ecosistema de Arduino.

Estas librerías se encargan de la comunicación, control mecánico y serialización de datos JSON enviados a la Raspberry Pi o la nube.

Librería	Función técnica	Equivalente en Raspberry Pi (Python)
WiFi.h	Gestiona la conexión a redes WiFi, configurando SSID y contraseña, obteniendo IP, reconectando en caso de fallo.	socket, wifi (en network o RPi.GPIO + subprocess si usa adaptador WiFi externo).
ESP32HTTPClient.h	Permite realizar peticiones HTTP/HTTPS (GET, POST) a APIs REST, con encabezados y cuerpo JSON.	requests, http.client
ArduinoJson.h	Serializa y deserializa objetos JSON para enviar y recibir datos estructurados.	json (módulo nativo de Python).

Servo.h	Controla servomotores mediante señales PWM precisas (posición angular de 0°–180°).	RPi.GPIO, pigpio, gpiozero.Servo.
----------------	--	-----------------------------------

En Raspberry Pi, el firmware Python puede integrarse con un ESP32 esclavo vía UART o I2C, donde el ESP32 controla motores y sensores, mientras la Raspberry se encarga de la IA y comunicación con la nube.

- El modelo TFLite se ejecuta localmente en la Raspberry Pi para clasificar imágenes capturadas. Su función es determinar si el residuo pertenece a una categoría específica (por ejemplo, reciclable / no reciclable).

Modelo	Descripción	Ventajas
MobileNetV2 (TFLite)	Red ligera optimizada para dispositivos embebidos, puede entrenarse para clasificar tipos de residuos.	Bajo consumo de CPU y memoria.
EfficientNet-Lite0	Variante eficiente compatible con TensorFlow Lite, buen equilibrio entre precisión y velocidad.	Ideal para Raspberry Pi 4 o superior.
ResNet50 (cuantizado TFLite)	Modelo robusto para clasificación más precisa, aunque más pesado.	Mayor exactitud en ambientes complejos.

Custom CNN (2 clases)	Modelo entrenado específicamente con dataset de residuos del proyecto.	Adaptado al contexto y materiales reales.
Teachable Machine (Google)	Permite generar rápidamente modelos TFLite binarios sin código.	Entrenamiento accesible y rápido.

- Backend AWS: Lambda, API Gateway, S3, RDS (MySQL)

El backend maneja el procesamiento, almacenamiento y sincronización de los eventos capturados por las Raspberry Pi.

Servicio AWS	Función técnica	Detalle de operación
AWS Lambda	Funciones serverless que procesan solicitudes sin necesidad de servidores.	Cada función se activa cuando el API Gateway recibe una solicitud (por ejemplo: /events, /image/:id, /stats). Maneja la lógica de negocio (guardar evento, generar URL firmada, consultar DB).
Amazon API Gateway	Gestiona los endpoints REST que recibe el frontend o los dispositivos IoT.	Valida el token JWT de Cognito, enruta la petición a la Lambda correspondiente, aplica límites de tráfico y autenticación.

Servicio AWS	Función técnica	Detalle de operación
Amazon S3 (Simple Storage Service)	Almacena imágenes o datos binarios enviados desde las Raspberry Pi.	Las Lambdas generan URLs firmadas (presigned URLs) que permiten subir/descargar imágenes de forma segura.
Amazon RDS (MySQL)	Base de datos relacional que guarda los registros de clasificación, usuarios, estadísticas y eventos.	Las Lambdas se conectan mediante el SDK de AWS para realizar operaciones SQL (INSERT, SELECT, UPDATE).

- Frontend: React.js + TailwindCSS

El frontend es la interfaz web utilizada por los administradores y usuarios para visualizar y gestionar la información del sistema.

React.js: Es una biblioteca de JavaScript basada en componentes, ideal para crear interfaces dinámicas y rápidas. Se emplea para desarrollar el panel administrativo que muestra los eventos enviados por las Raspberry Pi (fecha, tipo, imagen y estado), además de estadísticas, gráficos y configuraciones de usuarios o dispositivos.

Ventajas técnicas:

- Alto rendimiento gracias al uso del *Virtual DOM*.

- Estructura modular: cada parte del sistema (login, dashboard, estadísticas) se desarrolla como un componente independiente.
- Fácil integración con el backend mediante solicitudes HTTP (fetch o axios).
- Escalable y fácil de mantener a medida que el sistema crece.

TailwindCSS: Es un framework CSS basado en utilidades que permite crear interfaces modernas y personalizadas sin escribir mucho código de estilo. Se usa para diseñar la apariencia del panel, incluyendo tablas, botones y tarjetas.

Ventajas técnicas:

- Reduce el uso de CSS manual.
- Optimiza el tamaño del archivo final eliminando estilos no utilizados.
- Se integra fácilmente con React y otros frameworks de interfaz.

4. Diagrama De Arquitectura

El diseño propuesto (con Raspberry Pi, API Gateway, Lambdas, S3 y RDS) corresponde claramente a una arquitectura cloud serverless orientada a eventos, más formalmente conocida como: Arquitectura Serverless Orientada a Eventos (Event-Driven Serverless Architecture).

Es una arquitectura donde no existen servidores dedicados gestionados por el desarrollador, sino que los componentes del sistema se activan por eventos (acciones, cargas, peticiones o triggers) y escalan automáticamente según la demanda.

Características principales de la arquitectura

Característica	Cómo se refleja en el sistema
Serverless (sin servidores dedicados)	No administras servidores, todo se ejecuta en AWS Lambda, S3 y RDS.
Orientada a eventos	Cada acción (detección, subida, error) genera un evento que dispara una función Lambda.
Escalable automáticamente	Si llegan 10 o 1000 imágenes al mismo tiempo, AWS Lambda escala sin que se deba configurar nada.
Desacoplada	Cada módulo (registro, subida, dashboard, sincronización) opera de forma independiente.
Basada en microservicios	Cada Lambda cumple una única responsabilidad (Single Responsibility Principle).

Característica	Cómo se refleja en el sistema
Alta disponibilidad y resiliencia	AWS gestiona la redundancia, uptime y recuperación.
Bajo costo operativo	Cobra solo por ejecución, no por servidores activos.
Integración con servicios gestionados	S3, RDS, CloudWatch, Cognito y SNS funcionan como componentes externos integrados por eventos.

4.1. Diagrama general de arquitectura

El diagrama general de arquitectura tiene como objetivo mostrar cómo se integra todo el sistema, desde el firmware, hardware (Raspberry) hasta los servicios en la nube (AWS) y la interfaz web (Dashboard), incluyendo los flujos de datos y responsabilidades de cada módulo.

El sistema cuenta con tres capas principales:

Capa	Descripción	Elementos principales
Capa Física / Firmware	Donde ocurre la detección, clasificación y registro local.	Raspberry Pi, sensores, cámara, mecanismo de movimiento.
Capa de Nube (Cloud AWS)	Donde se almacenan y procesan los datos, imágenes y estadísticas.	S3, API Gateway, Lambda, RDS
Capa de Aplicación / Presentación	Donde los usuarios visualizan métricas y validan eventos.	Dashboard web, autenticación, métricas

4.1.1. Capa De Firmware

El firmware del sistema embebido (Raspberry Pi) controla:

1. Los sensores (ultrasonido, cámara, mecanismo de movimiento).
2. La lógica de clasificación (modelo IA).
3. El almacenamiento local.
4. La sincronización con la nube (AWS).
5. El manejo de errores.

Su propósito es detectar, clasificar, registrar y sincronizar residuos, funcionando offline si no hay conexión.

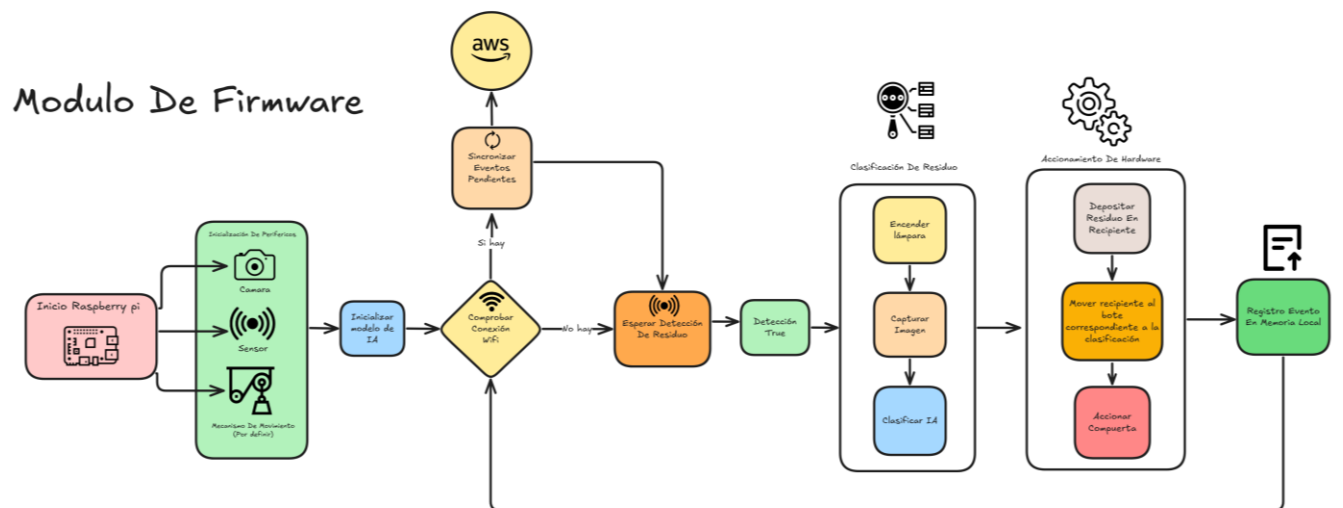


Figura 2. Diagrama De La Capa De Firmware

4.1.2. Capa De La Nube (AWS Reference Architecture)

Este módulo se encarga de:

- Recibir los datos enviados por la Raspberry Pi (eventos, imágenes, clasificaciones, errores).
- Almacenar imágenes, metadatos y logs.
- Exponer una API REST para la comunicación con el firmware y el dashboard web.
- Permitir la consulta, análisis y visualización de los datos.

Componentes Principales de AWS:

Componente	Función	Tipo de Servicio AWS	Comunicación
API Gateway	Expone la API REST segura hacia Raspberry Pi y Dashboard	Amazon API Gateway	HTTPS
Lambda Functions	Ejecutan la lógica backend (procesar eventos, guardar en BD, generar URLs pre-firmadas)	AWS Lambda	Interna (invocación por API Gateway o S3 triggers)
S3 Bucket	Almacena imágenes enviadas por la	Amazon S3	PUT (Raspberry) / GET (Dashboard)

Componente	Función	Tipo de Servicio AWS	Comunicación
	Raspberry mediante URLs pre-firmadas		
RDS Database	Guarda los metadatos, resultados de clasificación, y estado de sincronización	Amazon RDS (PostgreSQL)	SQL
CloudWatch	Registra logs, métricas y errores del sistema (tanto de Lambda como API)	Amazon CloudWatch	Interno
Cognito	Gestiona autenticación de usuarios del dashboard	Amazon Cognito	JWT Tokens
SNS	Cola de mensajes o alertas en caso de error o evento crítico	Amazon Simple Queue Service / Simple Notification Service	Asíncrono
IAM Roles	Control de permisos entre los servicios	AWS IAM	Interno

Componente	Función	Tipo de Servicio AWS	Comunicación
	(Lambda ↔ S3 ↔ RDS)		
Event Bridge	Disparar procesos post-subida, como verificación de imagen o envío de notificaciones	Amazon evento bridge	Interno

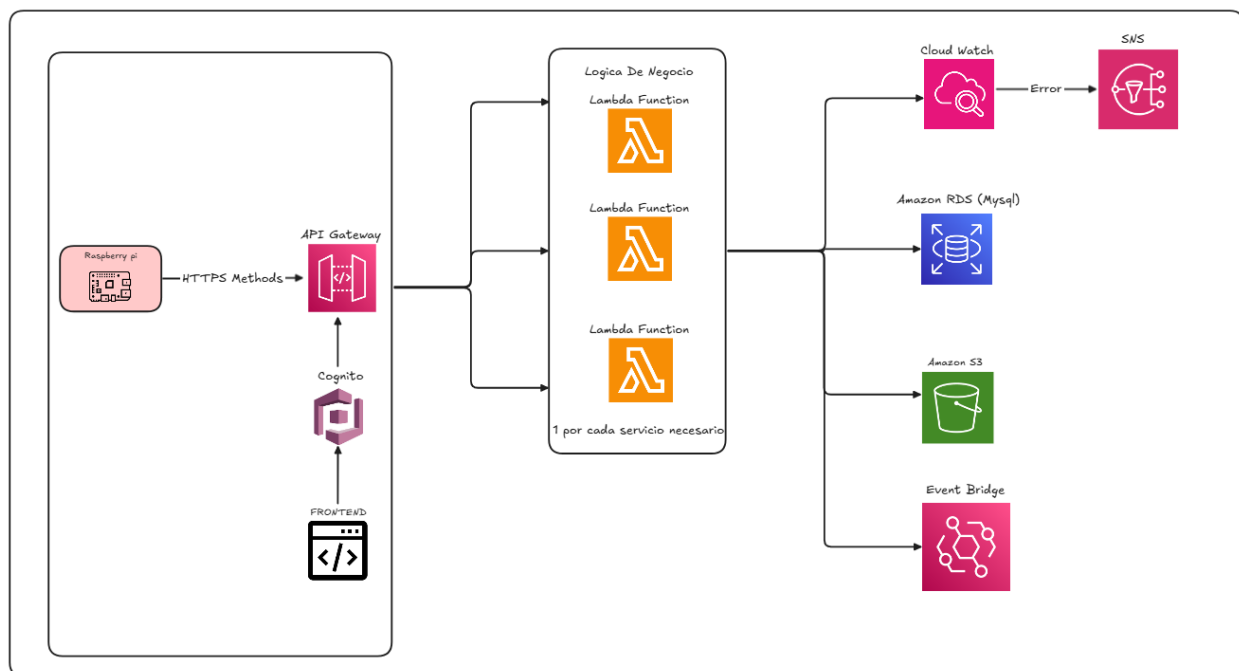


Figura 3. Diagrama De AWS Reference Architecture

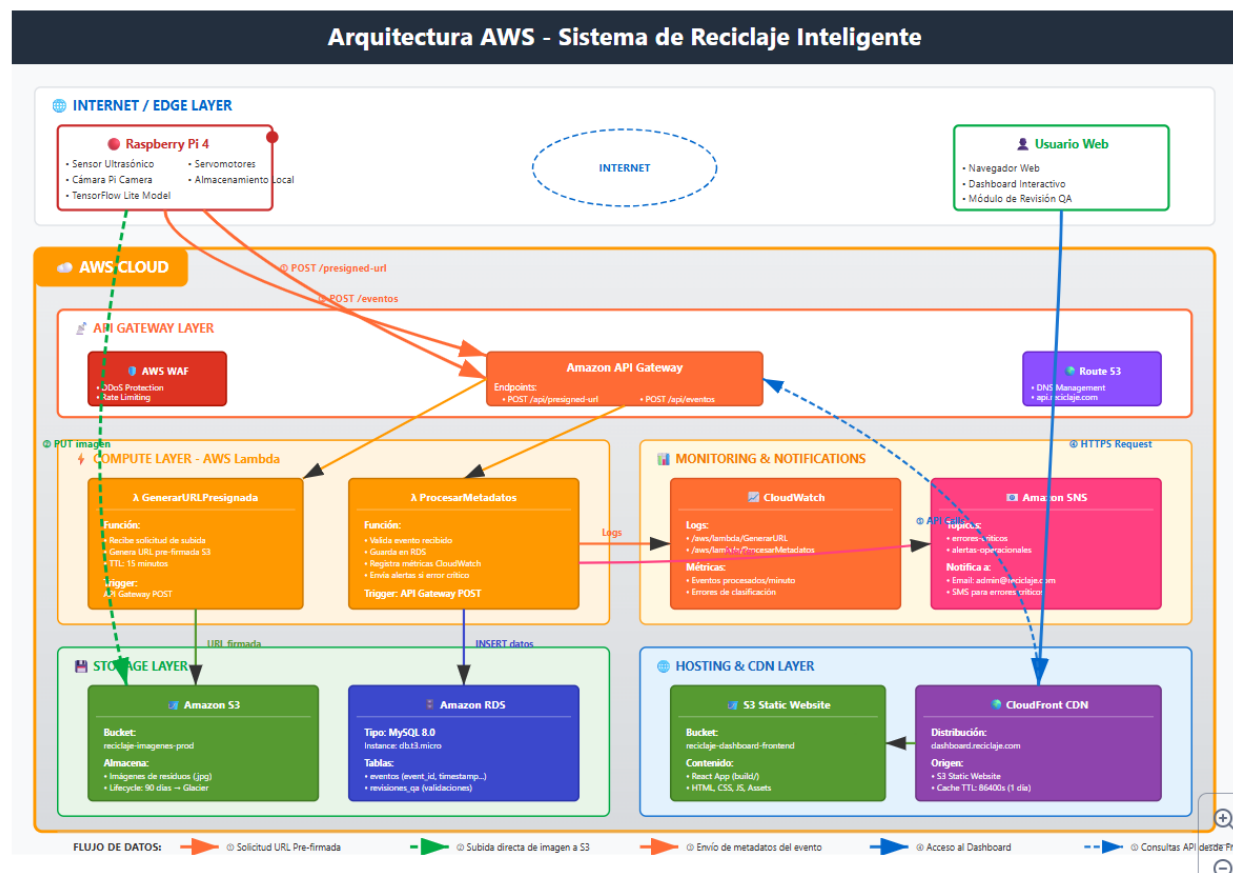


Figura 4. Diagrama De Arquitectura AWS

4.1.3. Capa De Presentación (Frontend)

El frontend será el dashboard administrativo y visual para:

- Consultar los eventos sincronizados por las Raspberry Pi.
- Visualizar imágenes y metadatos.
- Autenticar usuarios (usando Cognito).
- Gestionar validación de clasificaciones
- Mostrar estados, reportes y estadísticas.

Componentes Principales Del Frontend

Módulo	Función	Comunicación
Login / Registro	Autentica usuarios usando AWS Cognito	HTTPS (Cognito Hosted UI)
Panel Principal (Dashboard)	Muestra eventos sincronizados (fecha, imagen, tipo, estado)	HTTPS → API Gateway → Lambda → RDS
Módulo de Imágenes	Descarga o muestra imágenes almacenadas en S3	HTTPS → API Gateway → Lambda → S3 (GET presigned URL)
Módulo de Estadísticas	Consulta datos agregados (por fecha, tipo, dispositivo, etc.)	HTTPS → API Gateway → Lambda → RDS
Módulo de Configuración	Ajustes de usuario, notificaciones o parámetros de dispositivos	HTTPS → API Gateway → Lambda → RDS / SNS
Módulo de Alertas	Escucha notificaciones críticas (si usas SNS o EventBridge)	EventBridge (WebSocket / REST)

Flujo general de comunicación:

1. Usuario abre el dashboard web.
2. El frontend (React) carga y pide autenticación con Cognito.
3. Cognito devuelve un token JWT válido.
4. Con el token, el frontend llama al API Gateway (autorizado con Cognito).
5. API Gateway enruta las solicitudes a las Lambdas según el endpoint:

- /events → consulta lista de eventos.
 - /image/:id → obtiene URL firmada desde S3.
 - /stats → obtiene métricas.
 - /config → actualiza parámetros.
- Las Lambdas consultan la base de datos RDS o S3, procesan la información y devuelven la respuesta al frontend.
 - El frontend muestra los resultados en tablas, gráficas y vistas detalladas.
 - En caso de error → CloudWatch genera log → SNS o EventBridge notifica.

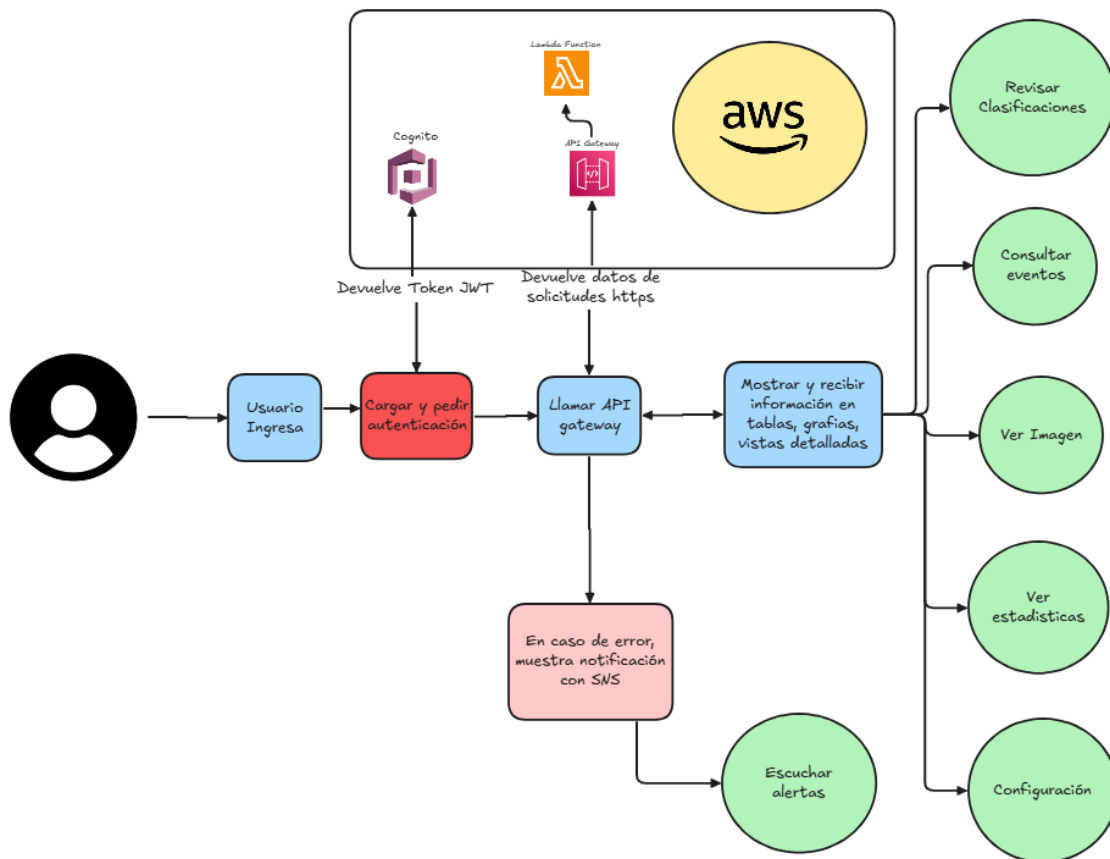


Figura 5.2 Diagrama De Arquitectura Del Frontend

Roles Identificados Para El Frontend

Rol	Permisos
Administrador	Puede ver todos los eventos, usuarios y configuraciones, además podrá revisar clasificaciones
Usuario	Solo ve eventos generales, no puede realizar operaciones como eliminar o modificar, solo podrá visualizar los datos.

4.2. Diagrama de casos de uso

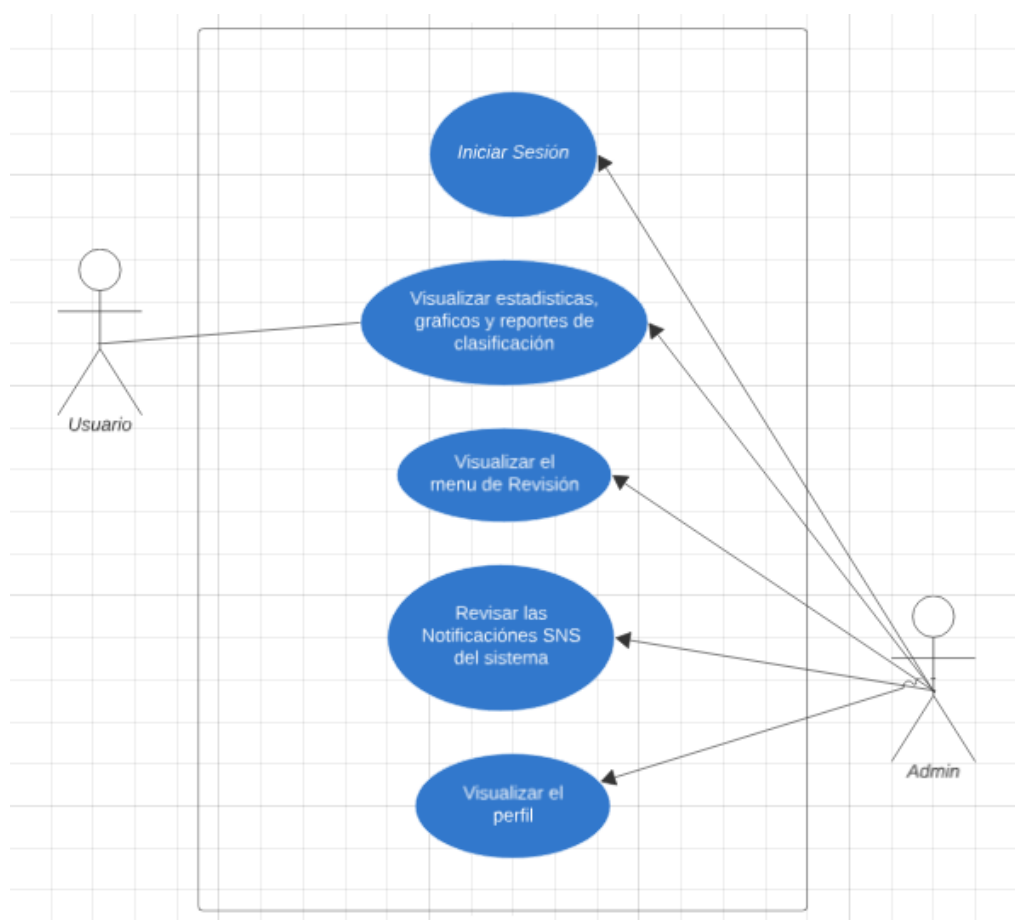


Figura 6.3 Diagrama De Casos de Uso

4.3. Diagrama de clases UML

En el presente proyecto, se debe realizar un diagrama de UML, en el que se representen las Clases que contendrá el proyecto, definiendo sus atributos, métodos y conexiones entre sí, para poder visualizar con exactitud como se orquestara la aplicación.

4.3.1. *Clases Identificables en el Proyecto*

1. Hardware/Firmware (Raspberry Pi)

- OrquestadorPrincipal: Es el "cerebro" del sistema. Coordina todos los componentes del hardware y ejecuta el ciclo completo: detectar → capturar → clasificar → actuar → guardar → sincronizar.
- SensorUltrasonico: Detecta cuando alguien coloca un residuo cerca del bote midiendo la distancia. Si detecta algo, activa todo el sistema.
- Cámara: Captura la foto del residuo para que pueda ser clasificado. También controla la iluminación para que la imagen salga bien.
- Lampara: Enciende una luz auxiliar para mejorar la iluminación cuando se va a tomar la foto del residuo.
- Mecanismo De Movimiento: Mueve las compuertas mecánicas para dirigir el residuo al compartimento correcto (reciclable o no reciclable).
- ClasificadorIA: Ejecuta el modelo de inteligencia artificial (TensorFlow Lite) que analiza la imagen y determina si el residuo es reciclable o no.
- ResultadoClasificación: Guarda el resultado de la clasificación: qué tipo de residuo es y qué tan seguro está el modelo (confianza).

- **EventoResiduo:** Representa un evento completo de clasificación. Guarda toda la información: ID único, hora, clasificación, imagen, errores, y si ya fue sincronizado con la nube.
- **GestorAlmacenamientoLocal:** Maneja el almacenamiento temporal en la Raspberry Pi. Guarda las imágenes y los eventos cuando no hay internet, y lleva un registro (index.json) de qué está pendiente por subir.
- **ClienteAPI:** Se comunica con AWS. Solicita URLs para subir imágenes a S3 y envía los metadatos (información del evento) a las funciones Lambda.
- **SincronizadorNube:** Supervisa la conexión a internet y sincroniza automáticamente los eventos pendientes cuando hay WiFi disponible. Si falla, reintenta.

2. Backend Serverless (AWS)

Aunque sean funciones Lambda, se pueden modelar como clases de servicio:

Clases principales:

- **API Gateway:** Es la "puerta de entrada" a tu backend. Recibe las peticiones HTTP de la Raspberry Pi y del Dashboard, y las dirige a las funciones Lambda correctas.
- **FuncionGenerarURLPresignada (Lambda):** Genera URLs temporales y seguras para que la Raspberry Pi pueda subir imágenes directamente a S3 sin necesitar credenciales de AWS en el hardware.
- **Funciones Procesamientos De Datos (Lambdas):** Reciben la información del evento (metadatos) desde la Raspberry Pi, la validan, y realizan alguna acción en el sistema (guardar, borrar, consultar). Si hay errores críticos, envía alertas.

- **DTOEvento (Data Transfer Object):** Es un "contenedor" que organiza los datos del evento para transferirlos entre sistemas. Asegura que la información tenga el formato correcto.
- **RepositorioEventos:** Es la capa que habla directamente con la base de datos. Realiza las operaciones: guardar eventos, consultar estadísticas, actualizar revisiones manuales.
- **ServicioNotificaciones:** Envía alertas cuando algo sale mal (errores críticos, sensores fallando, etc.) usando AWS SNS o CloudWatch. También registra métricas.
- **S3 Bucket:** Almacena todas las imágenes de los residuos clasificados. Es como un disco duro en la nube.
- **RDS (Base de Datos):** Almacena toda la información estructurada: eventos, clasificaciones, estadísticas, revisiones manuales. Es donde se guarda el historial completo.

3. Frontend/Dashboard (Web)

- **ControladorEstadisticas:** Maneja la lógica para mostrar las métricas y estadísticas en el dashboard: cuántos residuos se procesaron, porcentaje de reciclables, tiempos promedio, etc.
- **ControladorRevisionManual (QA):** Permite que los supervisores revisen las clasificaciones del sistema y las corrijan manualmente si el modelo se equivocó. Actualiza la base de datos con estas correcciones.
- **ServicioAPI (Cliente HTTP):** Es el intermediario entre el frontend y el backend. Realiza las peticiones HTTP al API Gateway para obtener datos o enviar actualizaciones.

- Estadísticas: Representa las métricas agregadas del sistema: totales, promedios, porcentajes. Es básicamente el "modelo de datos" para los gráficos del dashboard.
- Vistas/Componentes UI: Son las pantallas que el usuario ve: gráficos de barras, tablas con historial de eventos, formularios de validación manual, etc.

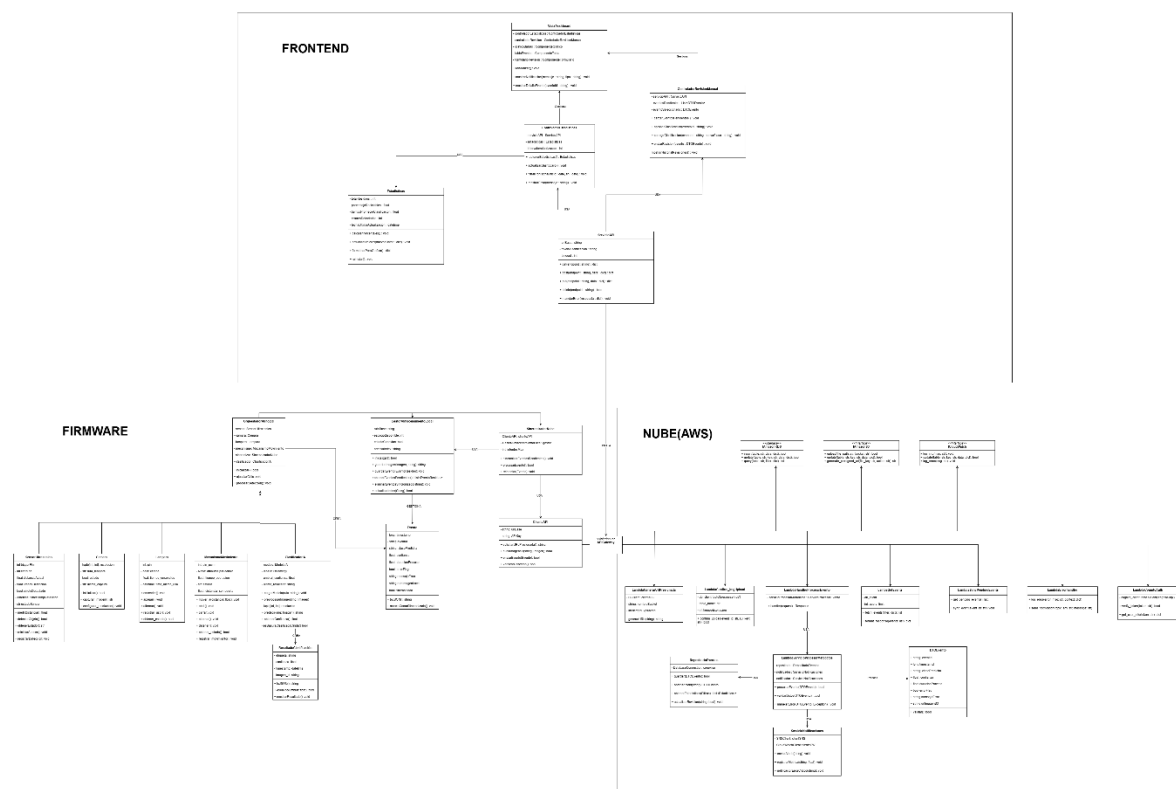


Figura 7.4 Diagrama UML Del Sistema Completo

La visualización del diagrama completo se puede ver desde:

<https://drive.google.com/file/d/1IDYkLefHMOy3xRvD-BgNobIsngx7hK5J/view?usp=sharing>

4.4. Diagrama ERD (Entidad-Relación)

Para el manejo de los datos generados por el dispositivo inteligente, se propone el diseño de una base de datos relacional que permita el registro, organización y administración eficiente de la información capturada durante el proceso de clasificación de residuos. Este enfoque facilita la integridad y consistencia de los datos, al tiempo que posibilita la trazabilidad de cada evento registrado, desde su detección en el hardware hasta su sincronización en la nube. Asimismo, el modelo relacional permite establecer relaciones entre entidades como eventos, administradores y registros de error, lo que favorece la consulta estructurada y el análisis posterior de métricas de desempeño, validación y precisión del sistema.

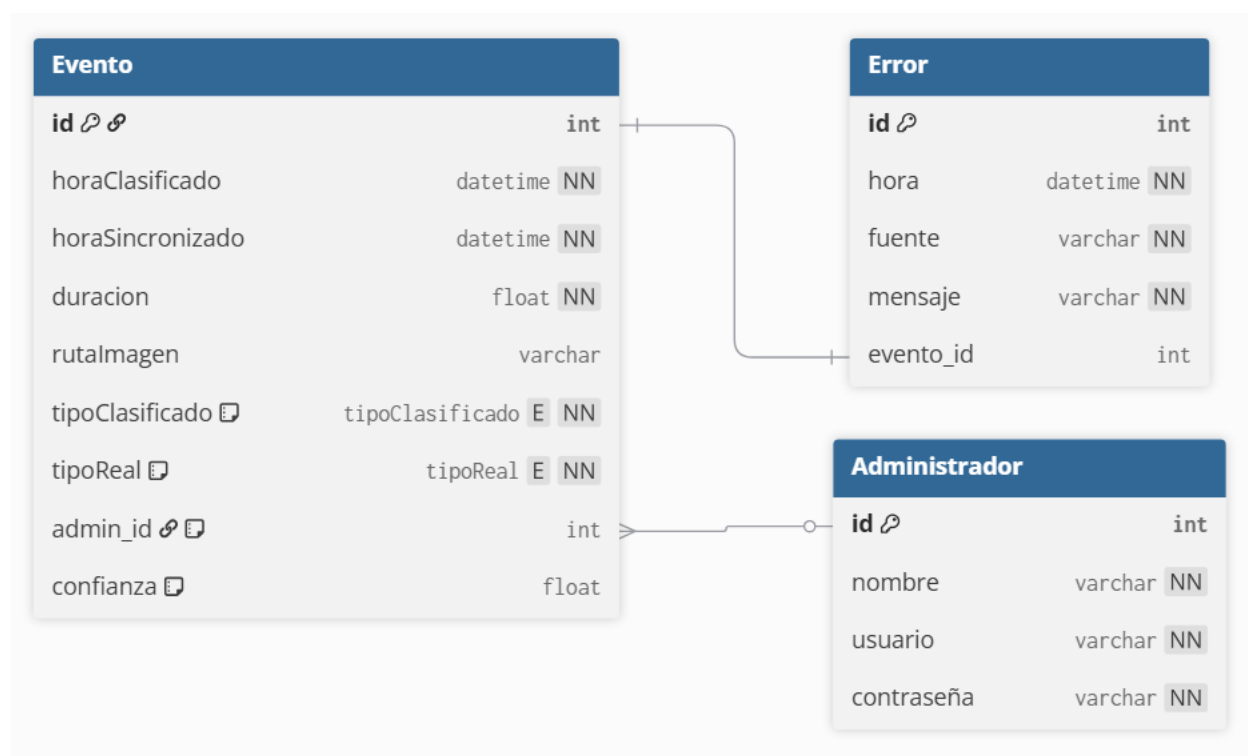


Figura 8. Diagrama Entidad Relación

La visualización detallada del diagrama, junto con su código base se puede realizar desde el siguiente enlace: <https://dbdiagram.io/d/ModeloBasura-68f5937a2e68d21b413aad23>

Relaciones observadas:

- Un Evento puede estar relacionado con un solo error, y cada Error puede pertenecer a un solo evento. (1:1). Si se elimina un evento, también se elimina su error (en cascada). Si cambia el ID del evento, se actualiza también en el error.
- Un Administrador puede revisar muchos Eventos, y cada Evento es revisado por un solo Administrador (1:N). Si se borra un administrador, sus eventos asociados quedan como nulos, perdiendo la asignación, pero no el registro. Si cambia el id del admin, se actualiza también en los eventos.

Notas para tomar en cuenta respecto a la tabla Evento:

- horaClasificado se refiere al timestamp del sensor de movimiento.
- horaSincronizado se refiere al timestamp de cuando el evento se agrega a la db en la nube.
- duracion float se calcula al agregar el evento a la db en la nube.
- rutaImagen varchar puede ser null si hay error en guardado.
- tipoClasificado y tipoReal se manejan como tipo de datos Enum.
- confianza se calcula automáticamente luego de que un Administrador edite tipoReal.

4.5. Diagrama De Actividades

El diagrama de actividades ilustra el flujo completo, no a bajo nivel de detalle, sino más bien presentando el paso a paso del funcionamiento del sistema de clasificación inteligente de residuos, representando las interacciones entre sus principales componentes: el hardware del

dispositivo IoT, el firmware o capa lógica de control, la infraestructura en la nube (AWS) y la plataforma web de administración y monitoreo.

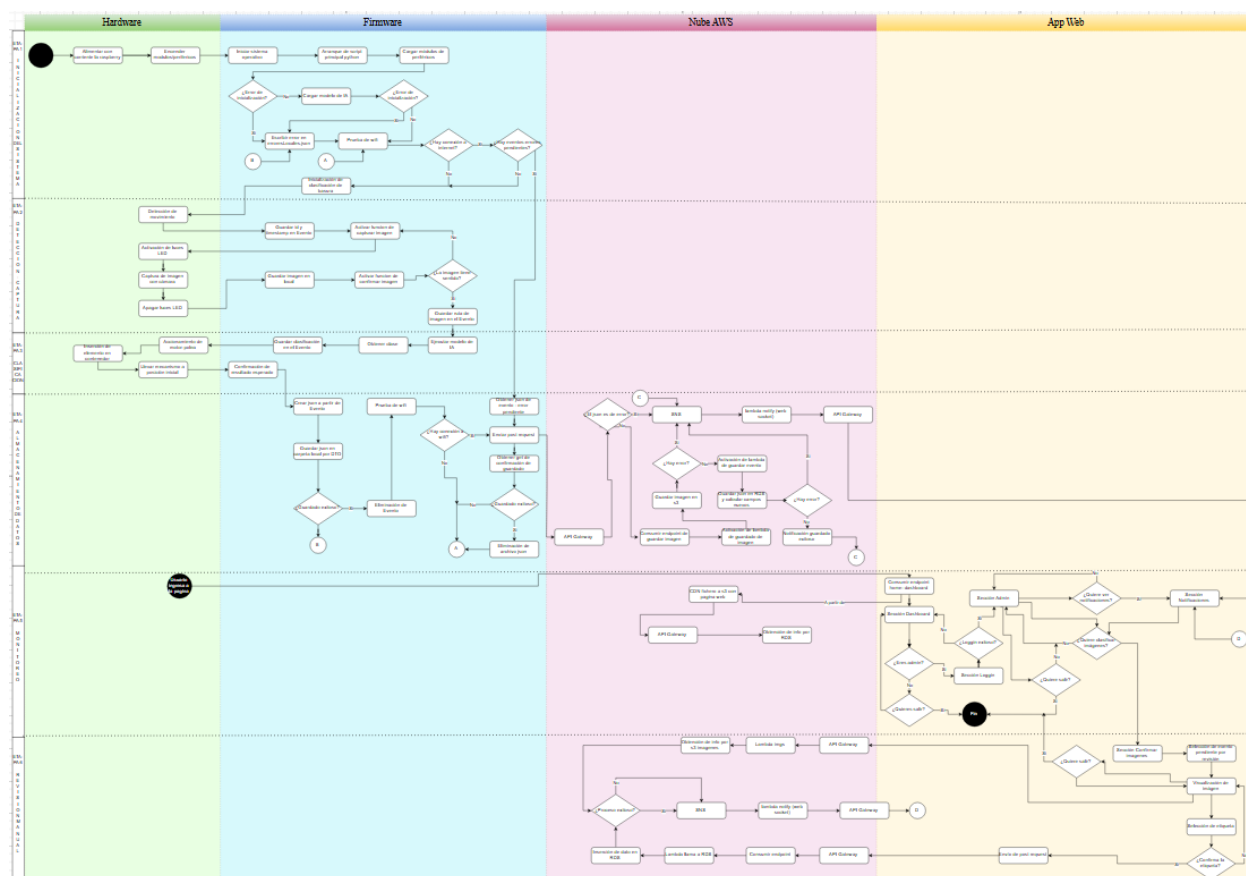


Figura 9. Diagrama de Actividades

La visualización detallada del diagrama, se puede evidenciar desde el siguiente enlace:

<https://dbdiagram.io/d/ModeloBasura-68f5937a2e68d21b413aad23>

Este diagrama se divide en cuatro carriles principales que reflejan las capas de procesamiento y comunicación del sistema:

- Hardware, que abarca la inicialización de los periféricos físicos (sensor ultrasónico, cámara, iluminación LED y servomotores), la detección de movimiento y la captura de la imagen del residuo.
- Firmware, donde se ejecuta el modelo de visión por computador para clasificar el tipo de residuo, se genera el evento correspondiente y se gestiona el almacenamiento temporal de datos en el dispositivo local, dentro de la raspberry, en sistema Linux, con Python.
- Sincronización con la nube (AWS), en la que los eventos clasificados son enviados a los servicios de AWS, como S3 para almacenamiento de imágenes, RDS para el registro estructurado de datos y Lambda/SNS para el manejo de notificaciones y errores, garantizando la integridad del flujo de información entre el contenedor físico y la infraestructura digital.
- Página web, en la cual los administradores o revisores QA, y los usuarios, acceden a una interfaz en línea que permite monitorear métricas, revisar clasificaciones manualmente, validar resultados y analizar errores registrados.

Además, el diagrama se estructura en seis etapas secuenciales, que representan el flujo lógico del proceso completo de funcionamiento del sistema de clasificación de residuos.

- Etapa 1: Inicialización del sistema

El dispositivo IoT (basado en Raspberry Pi) enciende y verifica el estado de sus periféricos, incluyendo los sensores, la cámara y el módulo de iluminación. También valida la conexión WiFi y carga el índice local de eventos pendientes, lo que garantiza que el sistema esté preparado para operar de forma autónoma o sincronizar datos previos con la nube.

- Etapa 2: Detección y captura

Comienza cuando el sensor ultrasónico identifica movimiento en el área de depósito. En respuesta, el firmware activa los LEDs para iluminar el objeto, y la cámara captura una imagen del residuo, que servirá como entrada para el modelo de clasificación.

- Etapa 3: Clasificación

Se ejecuta el modelo de visión por computador alojado localmente en la Raspberry Pi, el cual determina el tipo de residuo (reciclable, no reciclable u orgánico) y la predicción.

- Etapa 4: Almacenamiento de datos

El sistema genera un evento que incluye la imagen, los resultados de la clasificación y las marcas de tiempo correspondientes. Este evento se almacena temporalmente en un archivo local y, en caso de contar con conectividad, se sincroniza automáticamente con los servicios en la nube, mediante API Gateway, Lambda, S3 y RDS.

- Etapa 5: Monitoreo

Corresponde a la comunicación con la página web, donde se muestran los datos y se generan métricas de desempeño y notificaciones de error mediante el uso de servicios SNS. Permitiendo también a los administradores ver el estado del sistema en tiempo real.

- Etapa 6: Revisión Manual

Involucra la participación de revisores (QA) o administradores, quienes pueden validar o corregir las clasificaciones registradas desde la plataforma web, aportando retroalimentación que contribuye a mejorar la precisión del modelo y, permitiendo métricas reales sobre la gestión de residuos en la institución.

Es importante recalcar el funcionamiento del sistema de notificaciones, cuando el firmware envía un evento hacia la nube, este es recibido y evaluado por la Lambda principal, la cual determina el tipo de mensaje en función del contenido del evento o de su código de estado, así, si el procesamiento se realizó correctamente, se genera un mensaje de éxito; en caso de fallos, se emite un mensaje de error. Ambos tipos de mensajes se publican en un mismo tópico SNS (Simple Notification Service), utilizando atributos del mensaje, por ejemplo, `type: success` o `type: error`, para que los suscriptores puedan filtrarlos fácilmente según sus necesidades.

Entre los suscriptores del SNS se encuentra una segunda función Lambda, cuyo propósito es transmitir los mensajes al canal WebSocket. Esta Lambda actúa como intermediario entre SNS y los clientes web suscritos, tomando el contenido del mensaje y enviándolo al endpoint WebSocket activo. El servicio API Gateway (modo WebSocket) mantiene conexiones persistentes con los navegadores del dashboard, permitiendo que los administradores y revisores reciban actualizaciones inmediatas sin necesidad de recargar la página ni realizar solicitudes periódicas.

En conjunto, estas seis etapas conforman un flujo integral que abarca desde la detección física del residuo hasta la supervisión y validación humana, evidenciando la interacción entre el hardware, el firmware, la nube y la interfaz web como un ecosistema cohesivo.

5. Bocetos Página web

Los mockups diseñados representan la interfaz del dashboard de usuario y de administrador del sistema EcoVision USA, concebido como una plataforma web interactiva para la visualización y análisis de los datos generados por el contenedor inteligente de residuos. La estructura prioriza la claridad y la accesibilidad de la información, distribuyendo los elementos en secciones que facilitan la interpretación de métricas clave, como estadísticas puntuales, tasas de clasificación y evolución temporal del modelo.

En la parte superior, se incorpora un carrusel informativo destinado a mostrar avances del proyecto o mensajes institucionales, seguido de un conjunto de paneles que resumen indicadores de rendimiento del sistema. El área central se orienta al análisis visual, integrando gráficas dinámicas de clasificación a lo largo del tiempo y los registros más recientes de eventos procesados.

Finalmente, el diseño contempla un bloque de métricas específicas del modelo de IA, con descripciones sobre su comportamiento, precisión y limitaciones, complementado con un pie de página que agrupa información institucional, detalles del proyecto y canales de contacto. Así, el diseño propuesto responde a una lógica funcional y estética coherente con los principios de usabilidad, transparencia de datos y monitoreo continuo, asegurando una experiencia informativa integral tanto para administradores como para usuarios finales.

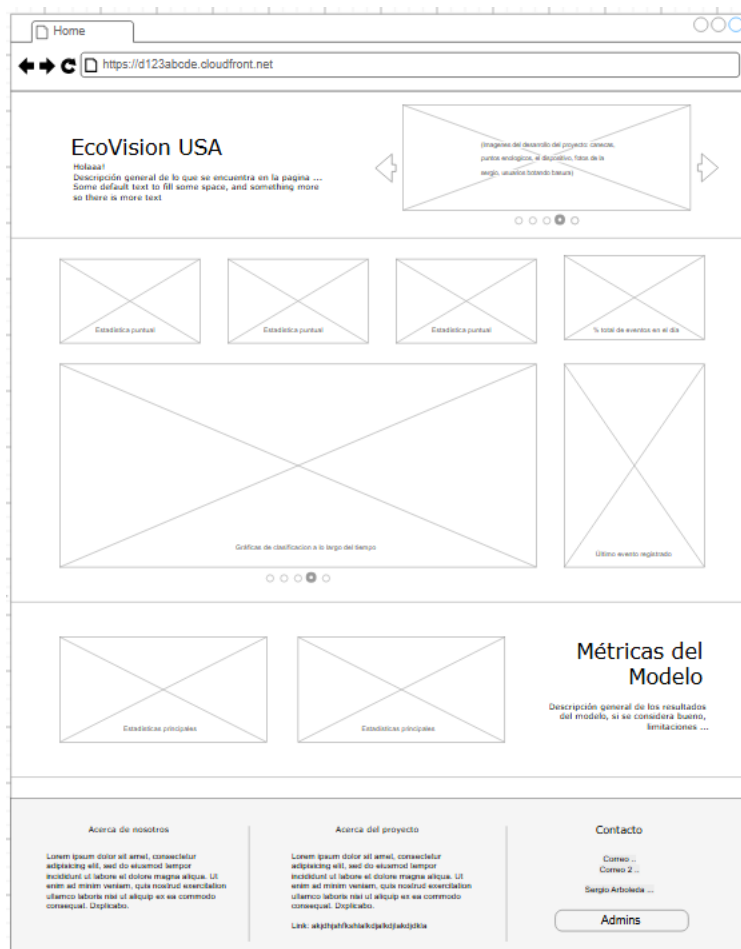


Figura 10. Mockup Pantalla Usuarios

Desde la página de usuarios, se dispone de un botón para que los administradores inicien sesión y entren a su panel personalizado, no se contará con módulo de registro, pues el registro se plantea quemando los datos en la base de datos.

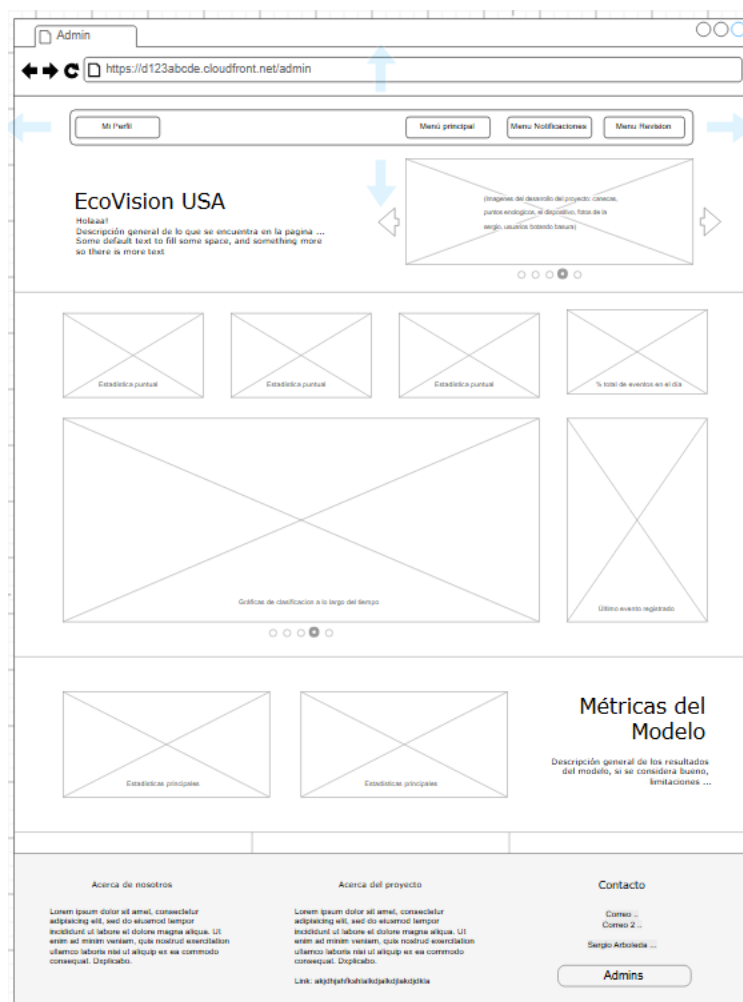


Figura 11. Mockup Pantalla Administradores

La visualización detallada de ambos mockups, se puede evidenciar desde el siguiente enlace:

https://drive.google.com/file/d/1JB2QsXwz2WWgk3YeZ_hAjNkPIUpD3UIF/view?usp=sharing

La página de administrador en sí es la misma que la del usuario, simplemente se agrega un header con un menú de acciones exclusivas del administrador, y al hacer click en el menú deseado, se redirigirá al endpoint correspondiente con la acción deseada.

Por otro lado, la paleta cromática seleccionada para la interfaz de EcoVision está compuesta por tonos fríos y armónicos que evocan limpieza, tecnología y sostenibilidad, conceptos directamente vinculados con la naturaleza del proyecto.

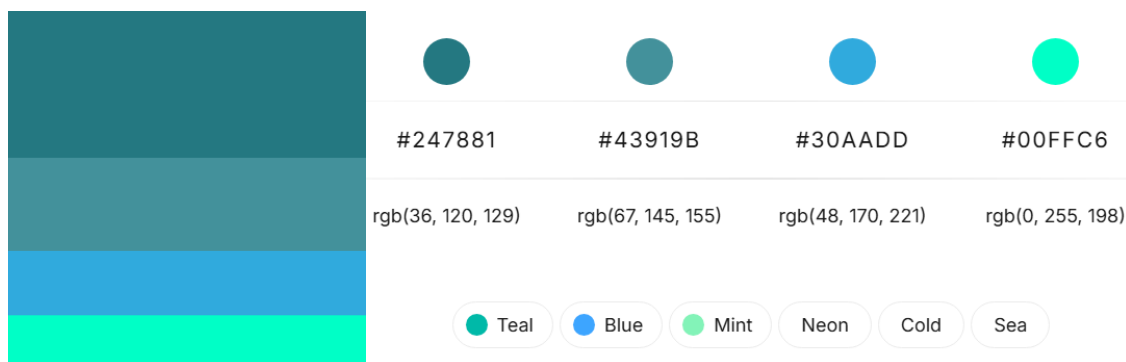


Figura 12. Paleta de Colores

Los matices de azul verdoso y turquesa predominan como base visual, representando tanto la pureza del entorno natural como la confiabilidad tecnológica del sistema. El azul más oscuro comunica estabilidad y profesionalismo, aportando contraste y jerarquía a los encabezados y menús principales, mientras que los tonos intermedios de azul y cian se asocian con innovación, frescura y conectividad digital.

Finalmente, el tono más claro de turquesa añade un acento energético que dinamiza la interfaz y refuerza la idea de acción ecológica y optimismo. La combinación de todos, logra un equilibrio entre lo funcional y lo emocional, proyectando una estética coherente con los valores de inteligencia ambiental, transparencia y compromiso sostenible que definen la identidad de la plataforma.

6. Cronograma Gantt

Con el fin de llevar a cabo un correcto desarrollo de la estructura planteada para el proyecto, se propuso una división de tareas por fases, además, las tareas tienen relacionado un participante del grupo en específico para ser realizada. Tienen relacionado también una fecha de comienzo y final de actividad, que permite un desarrollo continuo y afín con los tiempos de entrega del proyecto. A continuación, se evidencia el cronograma:

Indicador	Actividades	Responsable	Prioridad (Ruta Crítica)	# Sprint (SCRUM)	Inicio	Vencimiento	Duración (D)	Critica	Progreso
FASE 1: REQUISITOS									
1	Identificación de requerimientos generales	Tomas	Baja	1	19/03/2025	23/03/2025	1	Finalizado	100%
2	Realización de introducción	Tomas	Baja	1	19/03/2025	23/03/2025	1	Finalizado	100%
3	Definición de objetivo	Tomas	Baja	1	19/03/2025	23/03/2025	1	Finalizado	100%
4	Definición de alcance	Tomas	Baja	1	19/03/2025	23/03/2025	1	Finalizado	100%
5	Definición de público objetivo	Tomas	Baja	1	19/03/2025	23/03/2025	1	Finalizado	100%
6	Realizar flujo general de funcionalidades	Mateo, Valentina	Baja	1	19/03/2025	23/03/2025	1	Finalizado	100%
7	Identificar componentes generales del sistema	Mateo, Valentina	Baja	1	19/03/2025	23/03/2025	1	Finalizado	100%
8	Identificar requerimientos funcionales	Camilo	Alta	1	19/03/2025	23/03/2025	1	Finalizado	100%
9	Identificar requerimientos no funcionales	Camilo	Alta	1	19/03/2025	23/03/2025	1	Finalizado	100%
10	Identificar requerimientos de hardware	Tomas	Alta	1	19/03/2025	23/03/2025	1	Finalizado	100%
11	Plano de circuito	Tomas	Alta	1	19/03/2025	23/03/2025	1	Finalizado	100%
12	Identificar requerimientos de software (Selección de tecnologías)	Mateo, Valentina	Alta	1	19/03/2025	23/03/2025	1	Finalizado	100%
FASE 2: DISEÑO									
13	Definir la arquitectura general del sistema	Camilo	Alta	1	19/03/2025	23/03/2025	1	Finalizado	100%
14	Diagrama de Arquitectura del Sistema	Camilo	Alta	1	19/03/2025	23/03/2025	1	Finalizado	100%
15	Definir los roles en la web	Mateo, Valentina	Alta	1	19/03/2025	23/03/2025	1	Finalizado	100%
16	Diagrama de Casos de Uso	Mateo, Valentina	Alta	1	19/03/2025	23/03/2025	1	Finalizado	100.00%
17	Diagrama de UML	Camilo	Alta	1	19/03/2025	23/03/2025	1	Finalizado	100%
18	Diagrama Entidad-Relación (ERD)	Catalina	Alta	1	19/03/2025	23/03/2025	1	Finalizado	100.00%
19	Diagrama de Actividades	Catalina	Alta	1	19/03/2025	23/03/2025	1	Finalizado	100.00%
20	Diseño UI Mockup	Catalina	Alta	1	19/03/2025	23/03/2025	1	Finalizado	100.00%
FASE 3: DESARROLLO									
25	Planificación de desarrollo	Tomas	Baja	2	24/03/2025	29/03/2025	1	Pendiente	0.00%
Desarrollo de Red Neuronal									
26	Investigación de red neuronal preentrenada (MobileNetV2)	Tomas	Alta	2	22/03/2025	29/03/2025	4	Pendiente	0.00%
27	Adquisición y recolección de datos	Tomas	Alta	2	22/03/2025	29/03/2025	4	Pendiente	0.00%
28	Entrenamiento del modelo	Tomas	Alta	2	22/03/2025	29/03/2025	4	Pendiente	0.00%
29	Pruebas del modelo	Tomas	Alta	2	22/03/2025	29/03/2025	4	Pendiente	0.00%
30	Validación final (reentrenamiento o selección)	Tomas	Alta	2	22/03/2025	29/03/2025	4	Pendiente	0.00%
Desarrollo Frontend									
26	Configuración de carpetas	Valentina, Mateo, Camilo	Baja	3	24/03/2025	29/03/2025	6	Pendiente	80.00%
27	Instalación y entendimiento de tailwindCSS	Valentina, Mateo, Camilo	Baja	3	24/03/2025	29/03/2025	3	Pendiente	0.00%
28	Desarrollo diseño dashboard home	Valentina, Mateo, Camilo	Alta	3	24/03/2025	29/03/2025	4	Pendiente	0.00%
29	Desarrollo diseño dashboard admin	Valentina, Mateo, Camilo	Alta	3	26/03/2025	29/03/2025	2	Pendiente	0.00%
30	Configuración para consumo de API	Valentina, Mateo, Camilo	Alta	3	26/03/2025	29/03/2025	1	Pendiente	0.00%
31	Desarrollo de consumo de API	Valentina, Mateo, Camilo	Alta	3	26/03/2025	29/03/2025	1	Pendiente	0.00%
32	Pruebas de estado (probar endpoints y visualización)	Valentina, Mateo, Camilo	Alta	3	26/03/2025	29/03/2025	1	Pendiente	0.00%
33	Implementación de servicio WebSocket para notificaciones	Valentina, Mateo, Camilo	Alta	3	26/03/2025	4/04/2025	3	Pendiente	0.00%
34	Desarrollo de servicio de notificaciones	Valentina, Mateo, Camilo	Alta	3	26/03/2025	4/04/2025	2	Pendiente	0.00%
35	Desarrollo de servicio de autenticación	Valentina, Mateo, Camilo	Alta	3	26/03/2025	4/04/2025	2	Pendiente	0.00%
Desarrollo Backend AWS									
		Camilo, Catalina			24/03/2025	29/03/2025			

Figura 13. Diagrama de Gantt

La visualización detallada del cronograma, se puede evidenciar desde el siguiente enlace:

[GanttReciclajeInteligente.xlsx](#)